

Test results

Carlos Guido Gomez
Sergio Peralta Barboza
Julian Vargas Redondo

July 2026

1 Performance comparison

One chart per problem: execution time of all four algorithms as the instance size grows. The time axis is logarithmic (values still span several orders of magnitude) and is shown in the most readable unit per problem — milliseconds for the fast problems, seconds for the slow ones. A line that stops early means that method exceeded the 600 s cap.

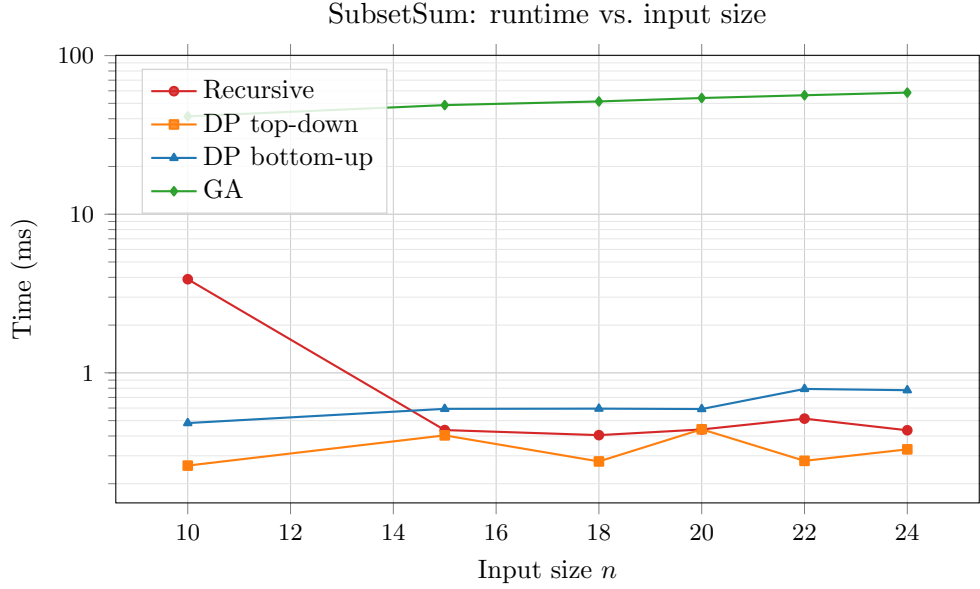


Figure 1: SubsetSum: execution time of each algorithm as the instance size grows (ms, log scale).

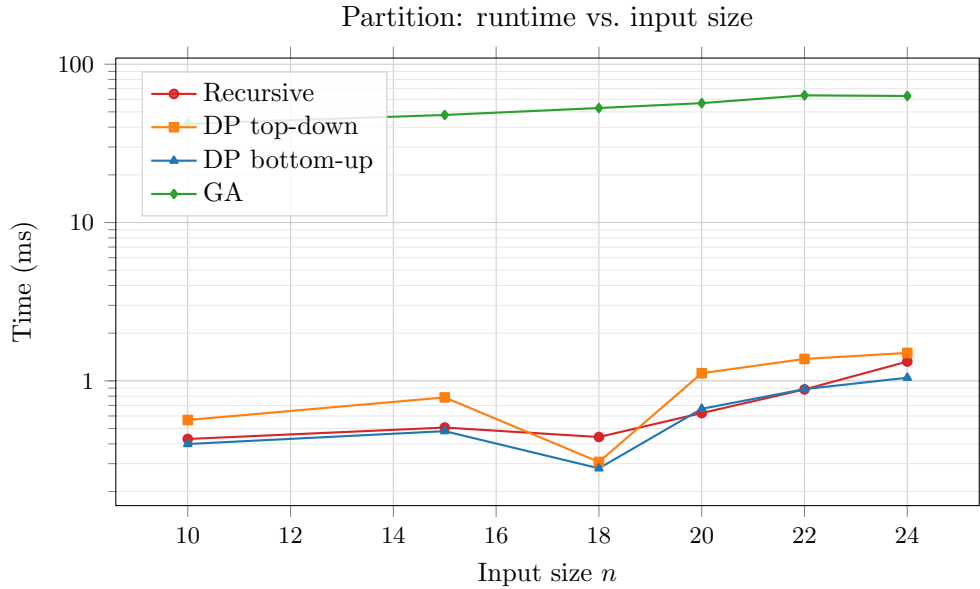


Figure 2: Partition: execution time of each algorithm as the instance size grows (ms, log scale).

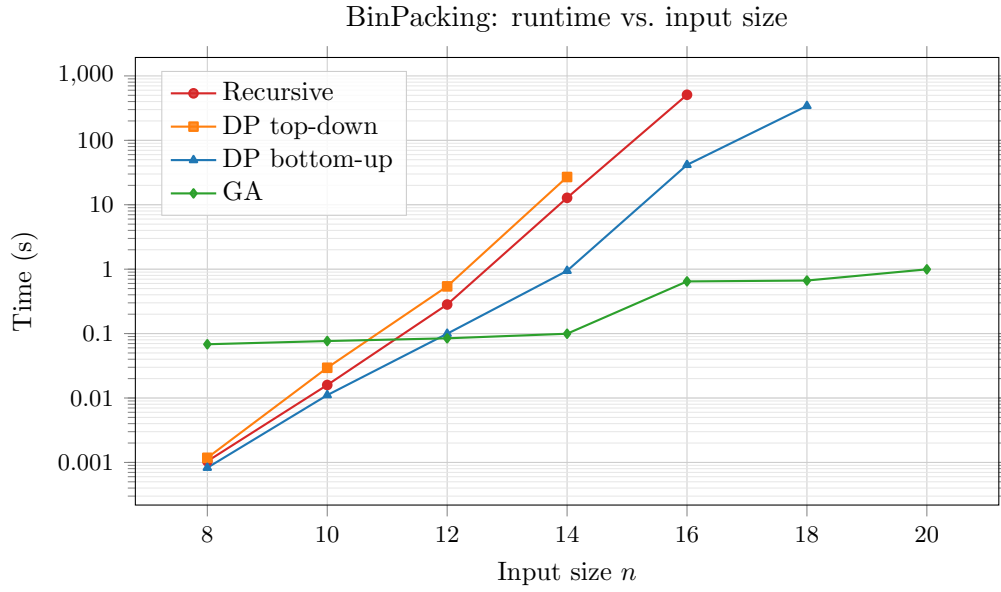


Figure 3: BinPacking: execution time of each algorithm as the instance size grows (s, log scale).

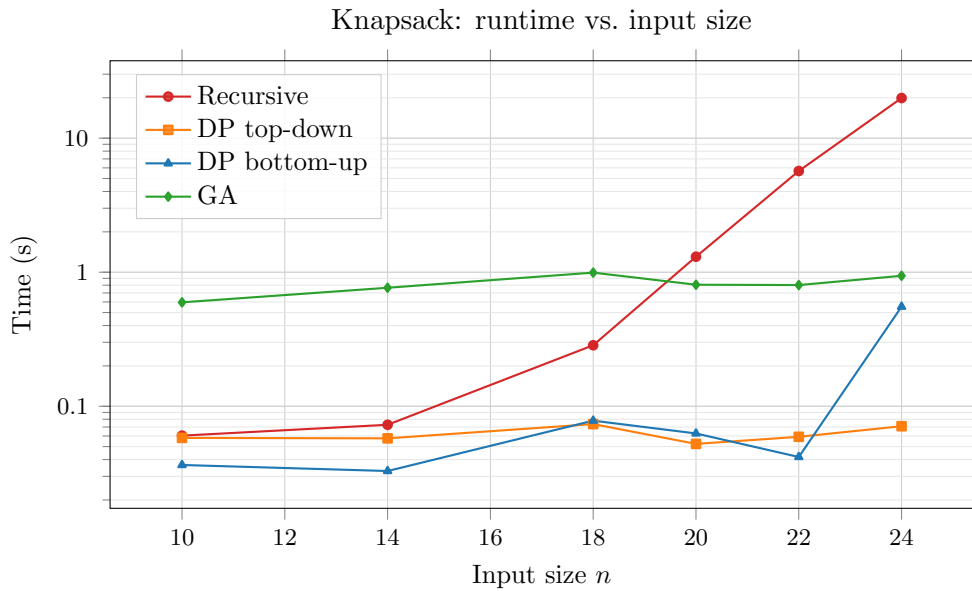


Figure 4: Knapsack: execution time of each algorithm as the instance size grows (s, log scale).

2 Interesting findings

Exact methods win small, GA wins big. On easy problems (SubsetSum, Partition) the exact methods run in a fraction of a millisecond, so GA is $\sim 100\times$ *slower* there. On BinPacking and Knapsack the exact methods blow up exponentially while GA stays roughly flat — GA only pays off once instances get large.

Too much mutation destroys Knapsack quality. Below 0.05 the GA always finds the same best value; at 0.1–0.2 mean fitness drops and variance explodes.

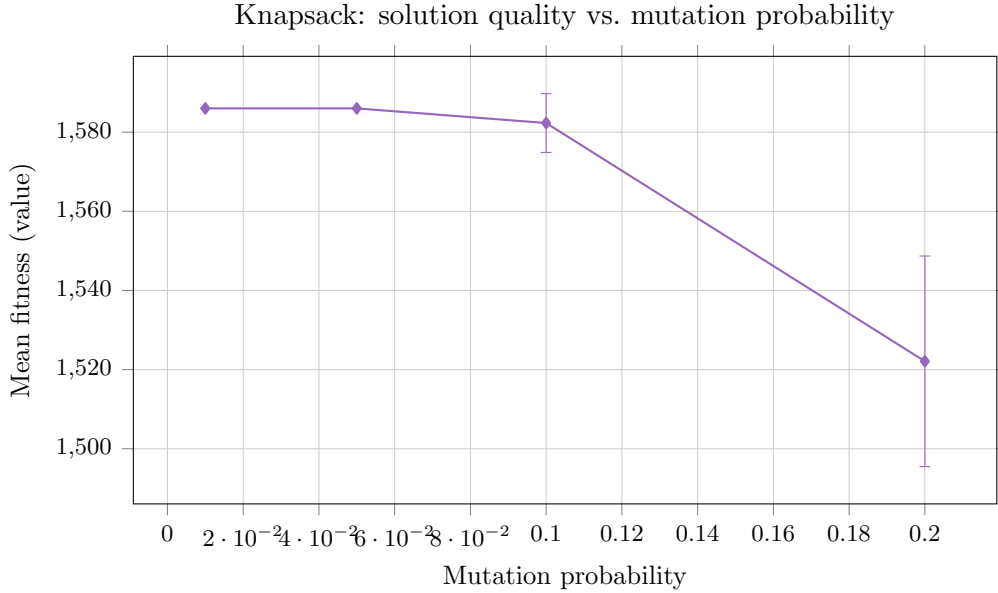


Figure 5: Knapsack mean best fitness vs. mutation probability (± 1 std, 10 runs).

Bigger population only buys cost, not quality. Runtime grows almost linearly with population size, while solution quality stays flat on these instances.

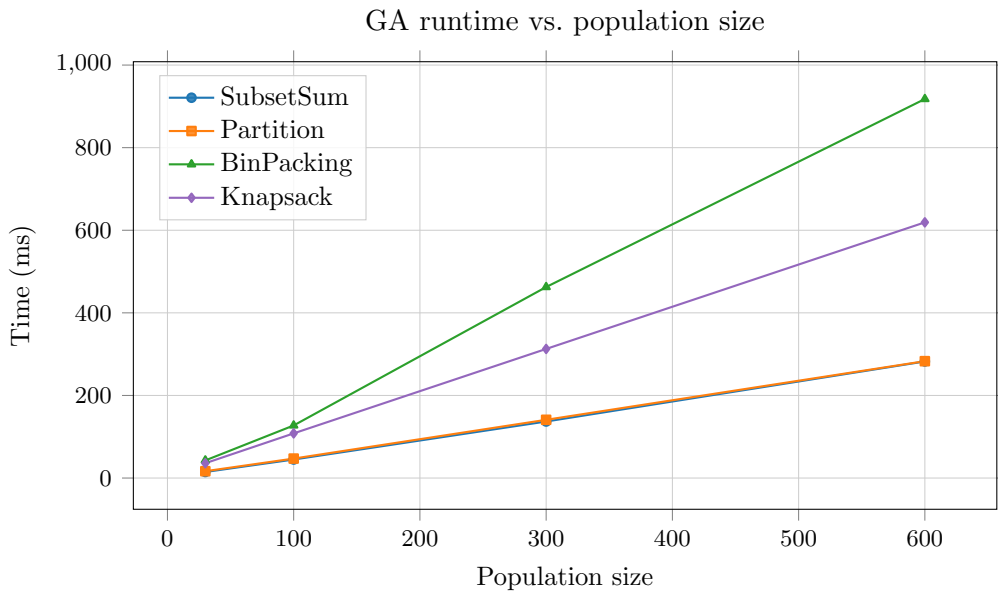


Figure 6: Mean GA runtime vs. population size (ms, 10 repeats each).

GA packs closer to optimal on larger instances. BinPacking's GA fitness ratio (1.0 = optimal) *improves* with size, from 1.33 down toward 1.13.

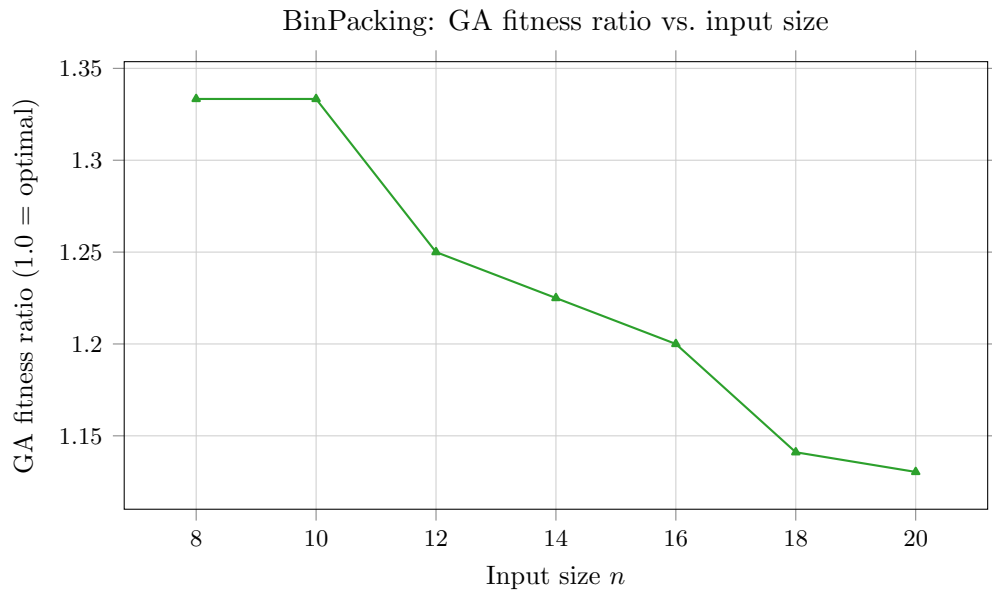


Figure 7: BinPacking GA fitness ratio; lower is better, 1.0 would be optimal.